CondoConecta

Plataforma Web de Gestão Condominial

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps: Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	CondoConecta (CDC)
Gerente de Projeto	Kaian Dias
Gerente/Responsável de GCS DevOps	Kaian Dias
Fábrica de Software / Squad	Individual, Atividade Avaliativa II (N1)
Repositório principal	https://github.com/KaianDias/condoconecta

Histórico de Revisões

Data	Versão	Descrição	Autor
22/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), a partir do template institucional.	Kaian Dias
28/10/2026	1.1	Inclusão do par de revisão com acesso de escrita, dos environments staging e production e da proteção da branch main.	Kaian Dias

Sumário

1. Introdução

2. Cultura DevOps e Papéis na Gerência de Configuração

3. Ferramentas, Ambientes e Infraestrutura

4. Identificação da Configuração

5. Integração Contínua (CI)

6. Entrega e Implantação Contínua (CD)

7. Infraestrutura como Código (IaC)

8. Controle de Configuração e Mudança

9. Baselines e Releases

10. Estimativa do Status de Configuração

11. Marcos

12. Treinamento e Recursos

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

14. Quadro-Resumo do Pipeline CI/CD

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto CondoConecta (CDC), com ênfase em práticas DevOps, especialmente em Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer o padrão a ser seguido no desenvolvimento individual do projeto CDC, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao desenvolvimento individual do sistema CondoConecta (CDC), uma plataforma web voltada à gestão de condomínios residenciais, permitindo que síndicos e moradores realizem reserva de áreas comuns (salão de festas, churrasqueira, quadra), publiquem e consultem avisos/comunicados, abram e acompanhem chamados de manutenção/ocorrências, participem de votações de assembleia e consultem o registro de encomendas/correspondências recebidas na portaria. O sistema conta ainda com um módulo de controle financeiro simplificado (status de pagamento da taxa condominial por unidade), um painel de controle (dashboard) com indicadores agregados para o síndico e um serviço de notificações automáticas por e-mail para eventos relevantes (novo aviso, confirmação de reserva, atualização de chamado). O autor deste documento é o único responsável por todas as atividades de desenvolvimento, operação e gestão de configuração do projeto, acumulando os papéis descritos na Seção 2.

Módulo	Descrição resumida
Reserva de áreas comuns	Calendário para reserva de salão de festas, churrasqueira, quadra, etc.
Avisos e comunicados	Publicação de avisos do síndico, visíveis a todos os moradores.
Chamados / ocorrências	Abertura e acompanhamento de chamados de manutenção.
Votação de assembleia	Registro de pautas e votação digital de moradores.
Encomendas	Registro de recebimento e confirmação de encomendas na portaria.
Financeiro	Status de pagamento da taxa condominial por unidade.
Dashboard do síndico	Indicadores agregados (reservas, chamados abertos, inadimplência).
Notificações por e-mail	Envio automático de e-mail para eventos relevantes do sistema.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que o desenvolvedor integra na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH), por exemplo 1.0.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
CDC	Sigla do projeto CondoConecta

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM), adaptado para um enfoque DevOps.
- Repositório do projeto: https://github.com/KaianDias/condoconecta
- Documentação oficial: GitHub Actions, Docker, Render (render.yaml / Blueprints).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega do CDC: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

Como o CDC é um projeto de desenvolvimento individual, todos os papéis típicos de uma squad DevOps são exercidos pelo mesmo autor. A tabela abaixo mantém a separação de responsabilidades apenas para fins de clareza processual. Na prática, o desenvolvimento, a operação e a gestão de configuração são feitos pelo mesmo autor, enquanto a revisão de código é feita por um par de revisão, Enzo Viana (usuário viaaanx no GitHub), que tem acesso de escrita ao repositório e aprova os Pull Requests antes do merge.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog (GitHub Issues/Projects), autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de branching e versionamento, administra o registro de artefatos/imagens (GHCR).
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados e é responsável pela qualidade do que é implantado.
SRE / Operações	Monitora o ambiente de produção (Render), acompanha health checks e realiza rollback quando necessário.
QA / Automação de Testes	Mantém a suíte de testes automatizados (Jest/Supertest) executada nos pipelines de CI e CD.
Revisor / Par de revisão	Enzo Viana (viaaanx), com acesso de escrita ao repositório. Revisa e aprova os Pull Requests antes do merge na branch main.
Segurança (DevSecOps)	Habilita análise de dependências (Dependabot) e verificação de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Papel representado pelo professor/orientador da disciplina, acionado apenas em mudanças de alto risco/impacto.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, IaC e pipelines como código
Orquestração de CI/CD	GitHub Actions	Automação de build, testes, análise e implantação
Empacotamento	Docker	Geração de imagens versionadas e imutáveis do backend e frontend
Registro de artefatos	GitHub Container Registry (GHCR)	Armazenamento e versionamento das imagens Docker

Infraestrutura como Código	Render Blueprint (render.yaml)	Provisionamento versionado dos serviços web e banco de dados
Hospedagem / Execução	Render (Web Service)	Execução dos ambientes de Homologação e Produção
Gestão de segredos	GitHub Actions Secrets	Armazenamento seguro de credenciais usadas nos pipelines
Monitoramento e Observabilidade	Render Metrics + UptimeRobot	Métricas básicas, logs e alertas de disponibilidade
Gestão de dependências / SCA	Dependabot + npm audit	Identificação de vulnerabilidades em bibliotecas de terceiros
Notificações	Resend (API de e-mail transacional)	Envio automático de e-mails para avisos, reservas e chamados
Gráficos / Dashboard	Chart.js	Visualização de indicadores agregados no painel do síndico
Rastreamento de tarefas	GitHub Issues + Projects	Vinculação de commits/PRs a itens de trabalho

3.2 Ambientes e Infraestrutura

Os ambientes do CDC são provisionados por meio de um arquivo render.yaml (Blueprint), garantindo paridade entre eles e permitindo recriação rápida e rastreável a partir do repositório.

Ambiente	**Finalidade**	**Gatilho de promoção**
Desenvolvimento	Execução local via docker-compose; validação individual antes do commit.	Manual, a cada ciclo de desenvolvimento
Homologação / Staging	Testes de aceitação e validação funcional da branch principal antes da produção.	Merge/push na branch main (pipeline de CI aprovado)
Produção	Ambiente utilizado pelos usuários finais do CDC.	Aprovação manual do gate de release após validação em staging, feita por Kaian Dias (revisor obrigatório do environment production)

Detalhamento: os ambientes de Homologação e Produção são serviços Web distintos no Render, cada um com sua própria URL (ex.: condoconecta-staging.onrender.com e condoconecta.onrender.com), banco de dados PostgreSQL isolado e variáveis de ambiente próprias, geridas via GitHub Actions Secrets e Render Environment Groups. No GitHub, esses ambientes são configurados como Environments chamados staging e production, e o environment production exige a aprovação de Kaian Dias como revisor obrigatório antes de cada deploy.

3.3 Estratégia de Branching e Versionamento

Adota-se Trunk-Based Development (GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main: sempre implantável (deployable), protegida por regra de repositório: o push direto é recusado e toda mudança entra por Pull Request aprovado pelo par de revisão.
- feature/<descrição-curta>: branches de curta duração (idealmente 1 a 2 dias) para cada mudança, como feature/reserva-de-areas-comuns.
- release/x.y (opcional): utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição>: correções emergenciais aplicadas diretamente em produção e depois reintegradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, os seguintes artefatos também são itens de configuração formais do CDC, versionados no mesmo repositório:

- Código-fonte do backend (Node.js/Express) e do frontend (React) e dos testes automatizados (Jest/Supertest), incluindo os módulos de financeiro, dashboard e notificações.
- Definições de pipeline como código (.github/workflows/ci.yml, .github/workflows/cd.yml).
- Dockerfiles (backend e frontend) e docker-compose.yml.
- Blueprint de infraestrutura (render.yaml).
- Arquivos de configuração por ambiente (.env.example com variáveis não sensíveis); os segredos reais ficam apenas no GitHub Actions Secrets.
- Scripts de migração de banco de dados (Prisma Migrate), cobrindo a modelagem de condomínios, unidades, moradores, reservas e chamados.
- Documentação técnica (README e ADRs, do inglês Architecture Decision Records).
- Este Plano de Gerenciamento de Configuração (PGCS), publicado em docs/PGCS_CDC_CondoConecta.pdf e versionado junto ao código, pois também é um item de configuração.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/reserva-de-areas-comuns
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(reservas): adiciona calendário de reserva do salão de festas
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0
Imagem de container	<registro>/<serviço>:<versão|sha>	ghcr.io/kaiandias/condoconecta-backend:1.0.0

4.3 Estrutura do Repositório

- /backend: API REST (Node.js/Express, Prisma ORM), testes em /backend/tests
- /frontend: aplicação React (interface web)
- /infra: render.yaml (Blueprint) e arquivos de configuração de ambiente
- /.github/workflows: pipelines de CI e CD (arquivos YAML)
- /docs: README, ADRs e este PGCS

5. Integração Contínua (CI)

Todo Pull Request para a branch main dispara automaticamente um pipeline de CI via GitHub Actions. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

- Checkout do código e restauração de dependências (cache de node_modules).
- Build da aplicação (backend e frontend).
- Análise estática de código (ESLint) e verificação de padrões de codificação.
- Testes unitários (Jest), com medição de cobertura mínima.
- Testes de integração (Supertest, contra banco de dados de teste em container).

- Análise de segurança de dependências (npm audit / Dependabot).
- Empacotamento e publicação da imagem Docker versionada no GHCR.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige a aprovação de pelo menos 1 revisor, o par de revisão Enzo Viana (viaaanx), que tem acesso de escrita ao repositório, antes do merge.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar a revisão.
- Merge somente permitido com pipeline de CI verde.
- A branch main é protegida: o push direto é recusado pelo GitHub, e a evidência dessa recusa fica registrada para verificação.

5.3 Critérios de Qualidade (Quality Gate)

- Cobertura de testes ≥ 70% nas rotas críticas (login, reserva de áreas comuns, abertura de chamados, votação de assembleia, lançamento de taxa condominial).
- Zero vulnerabilidades críticas/altas reportadas pelo npm audit.
- Zero erros de lint bloqueante (ESLint configurado como etapa obrigatória).

6. Entrega e Implantação Contínua (CD)

Após a publicação da imagem pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma imagem, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

- Deploy automático em ambiente de Homologação/Staging (Render, via Deploy Hook).
- Execução de testes automatizados de aceitação/E2E em staging.
- Gate de aprovação para produção: o environment production do GitHub exige Kaian Dias como revisor obrigatório antes do deploy.
- Deploy em produção com estratégia Rolling Update (Render).
- Testes de fumaça (smoke tests) pós-deploy e verificação de health checks (endpoint /health).
- Geração automática de tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Estratégia adotada no CDC: substituição gradual da instância no Render, minimizando indisponibilidade.
Blue-Green	Alternativa avaliada para versões futuras com maior criticidade, em que ambientes coexistem e o tráfego é comutado de uma vez.
Canary Release	Não aplicada na versão atual (custo de infraestrutura adicional); documentada como evolução futura.
Feature Flags	Utilizadas para funcionalidades experimentais (ex.: módulo de votação de assembleia), ativadas via variável de ambiente.

6.3 Rollback e Recuperação

Toda implantação do CDC é reversível: o pipeline de CD mantém a imagem anterior disponível no GHCR (tag versionada) e o histórico de deploys do Render, permitindo rollback manual imediato em caso de falha nos testes de fumaça ou nos health checks pós-deploy.

7. Infraestrutura como Código (IaC)

A infraestrutura dos ambientes de Homologação e Produção do CDC é descrita no arquivo infra/render.yaml (Render Blueprint), versionado no repositório e submetido ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

7.1 Versionamento de Infraestrutura

- Alterações no render.yaml seguem o mesmo processo de PR + pipeline; o Render aplica o Blueprint automaticamente após o merge na main.
- As variáveis de ambiente sensíveis não ficam no repositório; são geridas via Render Environment Groups e GitHub Actions Secrets.

7.2 Ambientes Efêmeros

Para Pull Requests que alteram o backend, um ambiente de revisão pode ser provisionado sob demanda (Render Preview Environments), permitindo validação isolada antes do merge, e é destruído automaticamente após o fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de Issue no GitHub → branch de feature → Pull Request → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Como o CDC é um projeto individual, o papel do CCM é representado pelo professor/orientador da disciplina, acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura (render.yaml), mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, o próprio pipeline de CI/CD substitui a aprovação formal do comitê.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção do CDC gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no GHCR e à revisão vigente do render.yaml.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas a partir dos Conventional Commits desde a última tag e publicadas junto à release no GitHub (seção Releases do repositório).

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção do CDC são monitorados continuamente via métricas nativas do Render e verificação externa de disponibilidade (UptimeRobot), permitindo identificar rapidamente falhas decorrentes de uma implantação.

10.2 Métricas DORA

Métrica	Descrição	Meta inicial no CDC
Deployment Frequency	Frequência com que o código é implantado em produção.	Ao menos 1 deploy por semana de desenvolvimento ativo
Lead Time for Changes	Tempo entre o commit e a implantação em produção.	Inferior a 1 dia
Change Failure Rate	Percentual de implantações que resultam em falha em produção.	Inferior a 15%
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.	Inferior a 1 hora (via rollback manual)

10.3 Relatórios e Auditorias

O histórico de execuções do GitHub Actions (builds, testes, deploys) e o log de commits/Pull Requests constituem, por si, o principal insumo de auditoria de GCS do CDC, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- Repositório criado e estrutura inicial versionada.
- Pipeline de CI implantado e validado (build + lint + testes).
- Pipeline de CD implantado em Homologação.
- Módulos de reserva de áreas comuns e chamados de manutenção implantados.
- Módulo financeiro (controle de taxa condominial) implantado.
- Dashboard do síndico e notificações por e-mail implantados.
- Primeira release automatizada em Produção (v1.0.0).
- Verificação do PGCS e do repositório em laboratório junto ao professor.

12. Treinamento e Recursos

Para implementar e manter as práticas de CI/CD, IaC e demais atividades de GCS descritas neste plano, o autor utiliza como base a documentação oficial das ferramentas adotadas:

- GitHub Actions: documentação oficial de workflows, secrets e environments.
- Docker: documentação oficial de build de imagens multi-stage.
- Render: documentação de Blueprints (render.yaml) e Preview Environments.
- Prisma ORM: documentação de migrations.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas de terceiros (npm) integradas ao CDC também são itens de configuração e são versionadas explicitamente via arquivo de dependências travado (package-lock.json).

- Análise automática de vulnerabilidades em dependências (Dependabot + npm audit) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente pelo Dependabot e submetidas ao mesmo fluxo de Pull Request.

Não há, na versão atual, software desenvolvido por subcontratados ou fornecedores externos; todo o código é de autoria própria, com exceção das bibliotecas de terceiros de código aberto listadas no package.json.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão do CDC, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de dependências, publicação da imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch main.	Pipeline de CI verde
CD (Staging)	Deploy automático em homologação, seguido de testes de aceitação.	Merge na branch main
Gate de Release	Aprovação manual para produção.	Sucesso dos testes em staging
CD (Produção)	Deploy por rolling update, smoke tests e geração de baseline/tag.	Aprovação do gate de release

Nota: este plano deve ser revisado ao longo do desenvolvimento do CDC, mantendo os quatro pilares da GCS (identificação, controle de versão, controle de mudanças e auditoria/status), sempre apoiados por automação.